rDataSet

The goal of rDataSet is to treat data frames as mathematical sets. It provides set-theoretic operations (union, intersection, difference, equality) for data frames where values are identified by row IDs and column names, and presence/absence is determined by `NA` values.

On top of Operations, Datasets support writing and reading to json, toml files. With the goal of having humand readable Data represntations which don't relay on table represnataions. (as there is a lack of decent diffing and viewing for such data, at least thats how i perceive it)

Installation

You can install the development version of rDataSet from local source like so:

```
devtools::install("/path/to/rDataSet")
# or with
install.packages("/path/to/rDataSet", repo = NULL, type = "source")
```

Overview

rDataSet introduces a `dataset` S3 class that extends data frames with designated ID columns. Set operations work on a cell-by-cell basis:

- **ID columns** uniquely identify each row (one-to-one relation)
- **Value presence** is determined by `is.na()` - a value is present if not `NA`
- **Set operations** compare cells at the same (row_id, col_id) position
- **Empty rows are dropped** - rows containing only `NA` values in all value columns are considered non-existent and are automatically removed during dataset construction

Set Operations

Each set operation works in two stages:

1. **Row/Column Alignment**: Determines which rows and columns appear in the result
2. **Value Operation**: Decides which values to keep for each cell

Operation	Rows in Result	Columns in Result
Difference (`A - B`)	All rows from A	All value columns from A
Intersection (`A > B`)	Only rows present in **both** A and B (inner join on IDs)	Only common value columns
Union (`A + B`)	All rows from both A and B (full join on IDs)	All value columns from both
Equality (`A == B`)	Must match exactly (same rows)	Must match exactly (same c

Note: ID columns must match between datasets for all operations. Value columns that exist in only one dataset are handled according to the join type (dropped, kept, or error).

Example

```
library(rDataSet)
library(tibble)
library(dplyr)
#>
#> Attaching package: 'dplyr'
#> The following objects are masked from 'package:stats':
#>
#>     filter, lag
#> The following objects are masked from 'package:base':
#>
#>     intersect, setdiff, setequal, union

# Create datasets with 'i' as the ID column
# Note: Rows with all-NA values in value columns are automatically dropped
A <- dataset_build(
  tibble(i = 1:10, b = if_else(1:10 %% 2 == 0, NA, 1:10), c = 1:10),
  ids = "i"
)

B <- dataset_build(
  tibble(i = 1:10, b = na_if(1:10, 3), c = 11:20),
  ids = "i"
)

# Dataset C has fewer rows (1:5) and an extra column 'd'
C <- dataset_build(
  tibble(i = 1:5, b = 10:6, d = 2),
  ids = "i"
)

# View the datasets
A
#> # A tibble: 10 × 3
#>        i     b     c
#>    <int> <int> <int>
#>  1     1     1     1
#>  2     2    NA     2
#>  3     3     3     3
#>  4     4    NA     4
#>  5     5     5     5
```

```
#>  6     6    NA     6
#>  7     7     7     7
#>  8     8    NA     8
#>  9     9     9     9
#> 10    10    NA    10
```

```
B
```

```
#> # A tibble: 10 × 3
#>        i     b     c
#>    <int> <int> <int>
#>  1     1     1    11
#>  2     2     2    12
#>  3     3    NA    13
#>  4     4     4    14
#>  5     5     5    15
#>  6     6     6    16
#>  7     7     7    17
#>  8     8     8    18
#>  9     9     9    19
#> 10    10    10    20
```

```
C
```

```
#> # A tibble: 5 × 3
#>       i     b     d
#>   <int> <int> <dbl>
#> 1     1    10     2
#> 2     2     9     2
#> 3     3     8     2
#> 4     4     7     2
#> 5     5     6     2
```

Set Difference

Keep values from A where B has `NA` (uses left join, so all rows from A):

```
A - B
```

```
#> # A tibble: 1 × 2
#>       i     b
#>   <int> <int>
#> 1     3     3
```

Difference with C shows how non-common columns are dropped:

```
A - C  # Only column 'b' remains (common), row ids 1:5 from A
```

```
#> # A tibble: 10 × 3
#>        i     b     c
#>    <int> <int> <int>
#>  1     1    NA     1
#>  2     2    NA     2
```

```
#>  3     3    NA     3
#>  4     4    NA     4
#>  5     5    NA     5
#>  6     6    NA     6
#>  7     7     7     7
#>  8     8    NA     8
#>  9     9     9     9
#> 10    10    NA    10
```

Set Intersection

Keep values from A where B has non-`NA` values (inner join on rows, common columns only):

```
A > B
#> # A tibble: 10 × 3
#>         i     b     c
#>     <int> <int> <int>
#>  1     1     1     1
#>  2     2    NA     2
#>  3     3    NA     3
#>  4     4    NA     4
#>  5     5     5     5
#>  6     6    NA     6
#>  7     7     7     7
#>  8     8    NA     8
#>  9     9     9     9
#> 10    10    NA    10
```

Intersection with C - note only rows 1:5 and column 'b' are kept:

```
A > C
#> # A tibble: 5 × 2
#>       i     b
#>   <int> <int>
#> 1     1     1
#> 2     2    NA
#> 3     3     3
#> 4     4    NA
#> 5     5     5
```

Set Union

Combine values from both datasets (full join on rows, all columns):

```
A + B
#> # A tibble: 10 × 3
#>         i     b     c
```

```
#>    <int> <int> <int>
#>  1     1     1     1
#>  2     2     2     2
#>  3     3     3     3
#>  4     4     4     4
#>  5     5     5     5
#>  6     6     6     6
#>  7     7     7     7
#>  8     8     8     8
#>  9     9     9     9
#> 10    10    10    10
```

Union with C - includes all rows (1:10) and all columns (b, c, d):

```
A + C
#> # A tibble: 10 × 4
#>        i     b     c     d
#>    <int> <int> <int> <dbl>
#>  1     1     1     1     2
#>  2     2     9     2     2
#>  3     3     3     3     2
#>  4     4     7     4     2
#>  5     5     5     5     2
#>  6     6    NA     6    NA
#>  7     7     7     7    NA
#>  8     8    NA     8    NA
#>  9     9     9     9    NA
#> 10    10    NA    10    NA
```

Set Equality

Compare datasets cell-by-cell (requires matching rows and columns):

```
A == A  # All TRUE (or NA for missing values)
#> # A tibble: 10 × 3
#>        i b     c
#>    <int> <lgl> <lgl>
#>  1     1 TRUE  TRUE
#>  2     2 NA    TRUE
#>  3     3 TRUE  TRUE
#>  4     4 NA    TRUE
#>  5     5 TRUE  TRUE
#>  6     6 NA    TRUE
#>  7     7 TRUE  TRUE
#>  8     8 NA    TRUE
#>  9     9 TRUE  TRUE
#> 10    10 NA    TRUE
```

```
A == B  # Shows which cells match
#> # A tibble: 10 × 3
#>        i b     c
#>    <int> <lgl> <lgl>
#>  1     1 TRUE  FALSE
#>  2     2 FALSE FALSE
#>  3     3 FALSE FALSE
#>  4     4 FALSE FALSE
#>  5     5 TRUE  FALSE
#>  6     6 FALSE FALSE
#>  7     7 TRUE  FALSE
#>  8     8 FALSE FALSE
#>  9     9 TRUE  FALSE
#> 10    10 FALSE FALSE
```

Equality fails with mismatched rows or columns:

```
A == C  # Error: rows and columns don't match
#> Error in dataset_equality(a, b): cols don't align
#>  cols left: ibc
#>  cols right: ibd
```

Helper Functions

```
# Extract ID columns
ids(A)
#> # A tibble: 10 × 1
#>        i
#>    <int>
#>  1     1
#>  2     2
#>  3     3
#>  4     4
#>  5     5
#>  6     6
#>  7     7
#>  8     8
#>  9     9
#> 10    10

# Extract value columns (non-ID columns)
vals(A)
#> # A tibble: 10 × 2
#>        b     c
#>    <int> <int>
#>  1     1     1
#>  2    NA     2
```

```
#>  3     3     3
#>  4    NA     4
#>  5     5     5
#>  6    NA     6
#>  7     7     7
#>  8    NA     8
#>  9     9     9
#> 10    NA    10
```

Mathematical Properties

The set operations satisfy the fundamental identity: `a = (a n b) u (a / b)`

```
# Verify: A == (A > B) + (A - B)
all(vals(A == (A > B) + (A - B)), na.rm = TRUE)
#> [1] TRUE
```

License

GPL (>= 3)